Mermaid flowcharts for perturbation design

Gary Welz

Preprint. Not peer-reviewed. Posted on Zenodo. Text and figures may change in subsequent versions.

A methods-oriented primer for investigators combining large language models, RegulonDB-class resources, and logic-style process charts

Gary Welz
Researcher, New Media Lab, CUNY Graduate Center
Email: gwelz@gc.cuny.edu
ORCID: https://orcid.org/0009-0005-7806-0892
Version 1.1 · June 28, 2026
Zenodo DOI: https://doi.org/10.5281/zenodo.20831781

Recommended hands-on before reading the methods sections

In any contemporary large language model, run the following prompt verbatim (paste as written):

> Use mermaid markdown format to make a flowchart of the Lac Operon and deliver it to me as an html file.

Inspect the returned HTML. Then issue a *second* prompt for **any other biological process** you use in the lab or in silico (pathway, signaling cascade, drug mechanism). The remainder of this draft assumes familiarity with how variable, detailed, and revisable such outputs are - and why reconciliation with curated data remains necessary.

Abstract

Planning genetic, pharmacological, or nutritional perturbations is easier when the investigator can state, in advance, which molecular levers are plausible and which readouts would discriminate competing mechanisms. Logic-style flowcharts - authored as *diagrams-as-code* (here using **Mermaid** markdown) - provide a lightweight, versionable complement to pathway databases, genome browsers, and machine-learning predictors. This methods paper distills a seminar-tested workflow for a general research audience: we motivate text-based diagrams in wet-lab and computational pipelines; we compare three deliberately different encodings of the classical *Escherichia coli* lactose (*lac*) operon - a **literature-first logic diagram** with explicit Boolean-style gates, a **RegulonDB-emphasis regulatory wiring diagram** without those gates, and a **hybrid diagram** that retains interpretive logic while adding database-grounded entities (notably explicit *lacI* expression); we explain why the full two-input induction condition (inducer present and catabolite repression relieved) appears

only in some encodings; and we recommend *layered hybridization* (regulatory backbone + literature logic + identifier and parts audits) instead of naively merging incompatible ontologies. No bespoke software is required beyond ordinary LLM access, Mermaid rendering, and public databases.

---

1. Introduction

The *lac* operon remains the canonical bacterial example of integrating environmental signals, transcription factors, and promoter logic.1 Contemporary laboratories rarely study it in isolation, but it is ideal as a *tutorial system*: many curated representations exist, so **source choice** becomes visible as scientific information rather than as an invisible preprocessing step.

The idea that regulatory DNA encodes logical operations is not new to synthetic biology: Voigt et al. (2016) demonstrated that human-readable logic programs can be compiled directly into novel DNA sequences that execute gate-level circuits inside living bacterial cells.11 GLMP addresses the complementary problem — reading and decoding the logical structure already encoded in natural regulatory DNA, using the same grammar of AND, OR, and NOT gates that synthetic circuits are built from.

Meanwhile, perturbation-forward workflows - CRISPR screens, chemical genetics, single-cell readouts, virtual-cell-style predictors - benefit from an explicit, criticizable sketch of **inputs, branch points, feedback, and candidate measurements** before budget is committed. A flowchart is not a mechanistic ODE model and not a trained deep network; it is a **hypothesis artifact** suitable for group review, supplementary files, and teaching.

Mermaid is a widely supported markdown-adjacent language for flowcharts and related diagrams; source text diffs cleanly in Git and renders in browsers, notebooks, and static site generators.2 Modern LLMs can emit fenced `mermaid` blocks from natural language, which accelerates first drafts but *increases* the obligation to validate against authoritative resources: prompt and model choice change topology and node identity.

---

2. Background and rationale

High-dimensional expression or chromatin data and modern predictors answer different questions than a hand-specified logic chart. The chart's job is to make **conditional structure** discussable: which environmental axes gate which genes, where redundancy may hide causal effects, and which branch-point perturbations would be most informative. It should complement - not replace - controlled experiments and quantitative models.

Mermaid is practical in collaborative research groups for three reasons:

- **Text to vector diagram:** the same source renders in CI documentation, lab wikis, and supplementary PDF pipelines.
- **Collaboration:** pull requests on diagram source are interpretable; meeting-driven edits are quick.
- **LLM interface:** models can propose structure; human authors remain responsible for citations, organism fit, and experimental feasibility.

---

3. Objectives for the practicing investigator

- State **what could change** under a planned intervention before locking readouts or model architecture.
- Maintain **diagrams-as-code** beside literature notes and analysis scripts.
- Combine **LLM-assisted layout** with **RegulonDB-class** regulatory facts without collapsing distinct abstraction levels into one unreadable graph.
- Treat **disagreement between diagram sources** as data - analogous to comparing alignment or quantification pipelines.

---

4. LLM-first *lac* charts: what to expect

A well-posed conversational prompt often yields a surprisingly detailed first pass: two-input logic (allolactose / LacI and glucose / cAMP-CRP), default OFF at the operator, graded induction, sometimes a feedback arc as inducer is consumed. Reliability varies by model version and prompt; treat any auto-generated chart as **revision zero** to be checked against reviews and databases.

---

5. Three encodings of the same operon

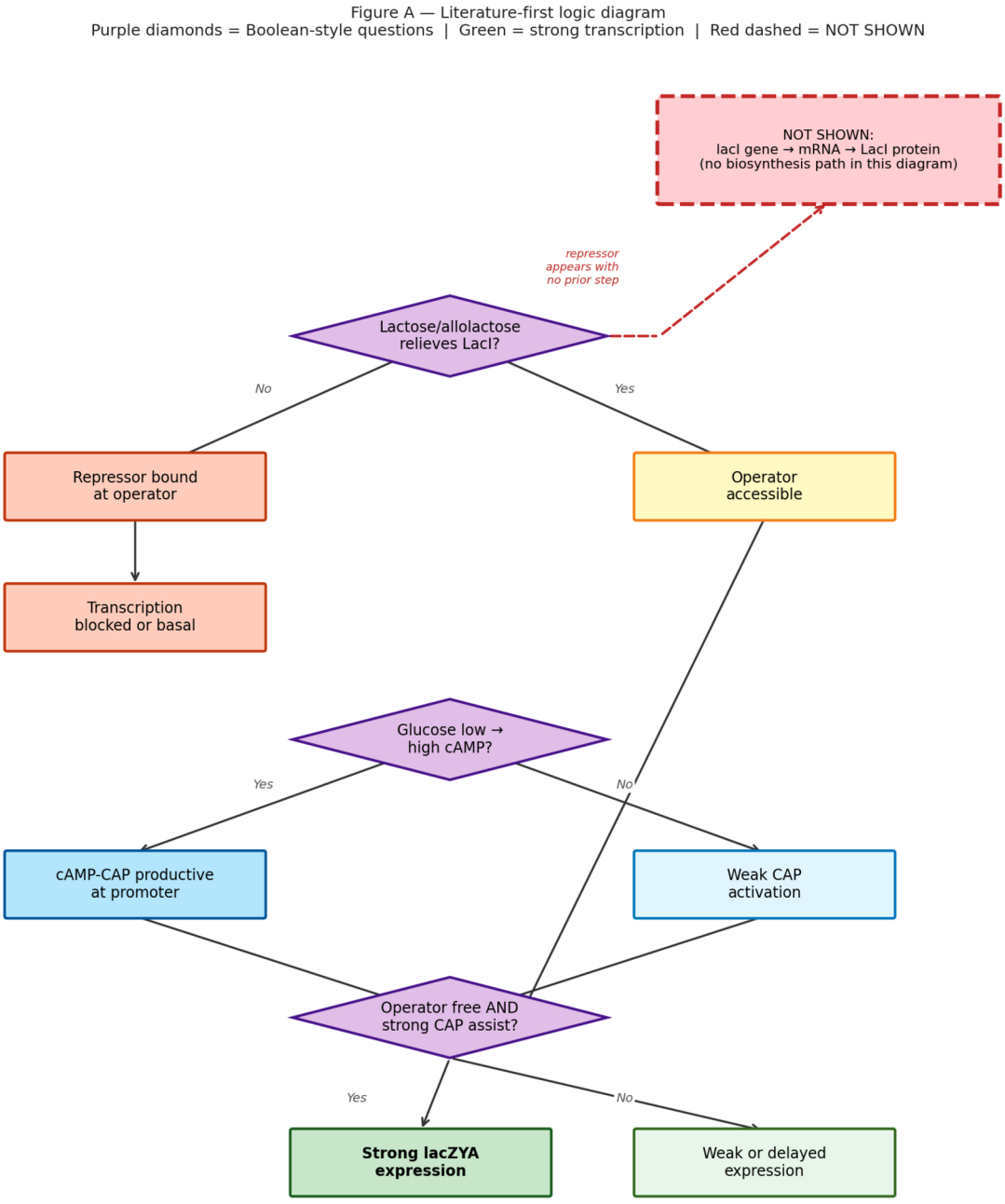
GLMP-style public comparisons (see viewer in Data Availability) contrast multiple evidence mixes for one operon. For methods exposition we isolate **three** complementary styles; names here are descriptive only.

5.1 Literature-first logic diagram

Built from textbook-style reasoning, this encoding foregrounds **explicit AND-style integration** (e.g. strong transcription when the operator is free *and* CAP is productively engaged) and separates high-glucose / low-cAMP branches. It is optimized for *pedagogy and perturbation intuition*, not for locus tags.

What is typically *missing* here: there is **no** first-class node for ***lacI* transcription** or for the **gene to protein** production of the repressor. LacI appears only inside the wording of the first decision node - the repressor is *assumed* to exist whenever lactose logic is discussed. That is fine for high-level logic, but it hides a real experimental lever (CRISPRi on *lacI*, titration of repressor copy number, etc.).

Figure A. Literature-first logic. Purple diamonds = explicit Boolean-style questions. Green = strong transcription outcome. Red dashed box (top right) = what this encoding leaves implicit: no biosynthesis path for LacI protein.



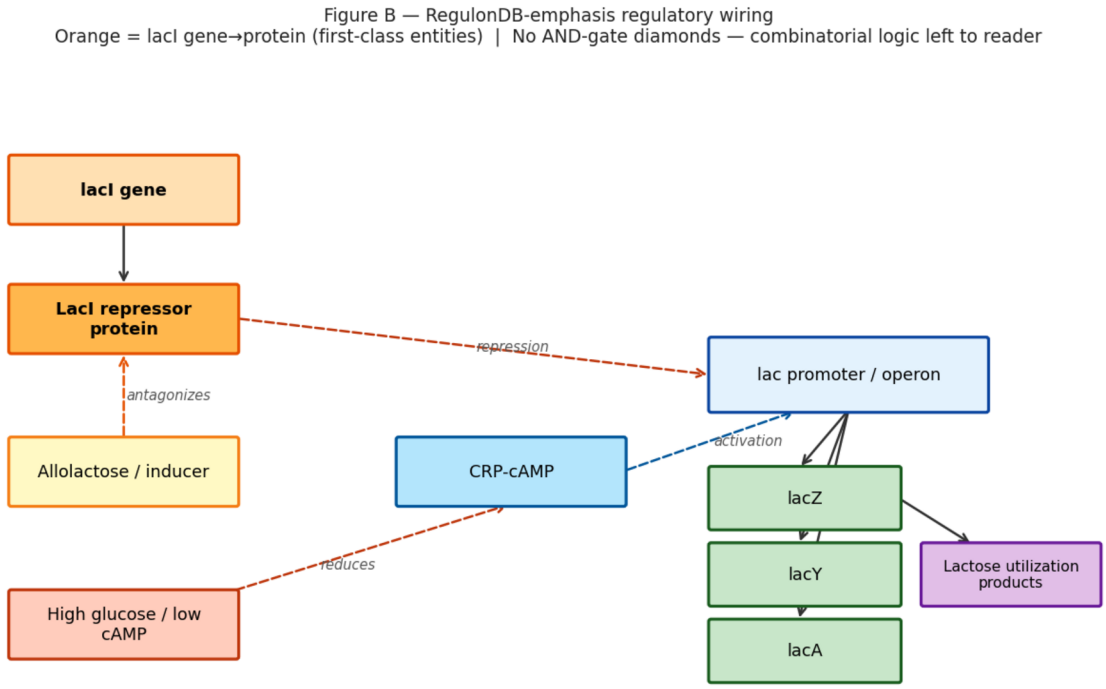
Read Figure A against Figure C: the red dashed annotation has no counterpart in the main flow - the hybrid instead inserts a real chain of orange nodes for *lacI* expression before any "repressor off operator?" decision.

5.2 RegulonDB-emphasis regulatory wiring

A chart faithful to how **RegulonDB** (and similar TF to target resources) represent *E. coli* transcriptional regulation centers **who regulates whom** with signed or labeled edges; it does not

encode the full Boolean "both conditions" story as explicit gate nodes.3 Figure B highlights the *lacI* gene to protein spine (orange) as a first-class entity; there are no purple AND diamonds.

Figure B. Regulatory wiring emphasis. Orange = *lacI* gene to protein (RegulonDB first-class entities). No AND-gate diamonds; combinatorial logic is left to the reader.



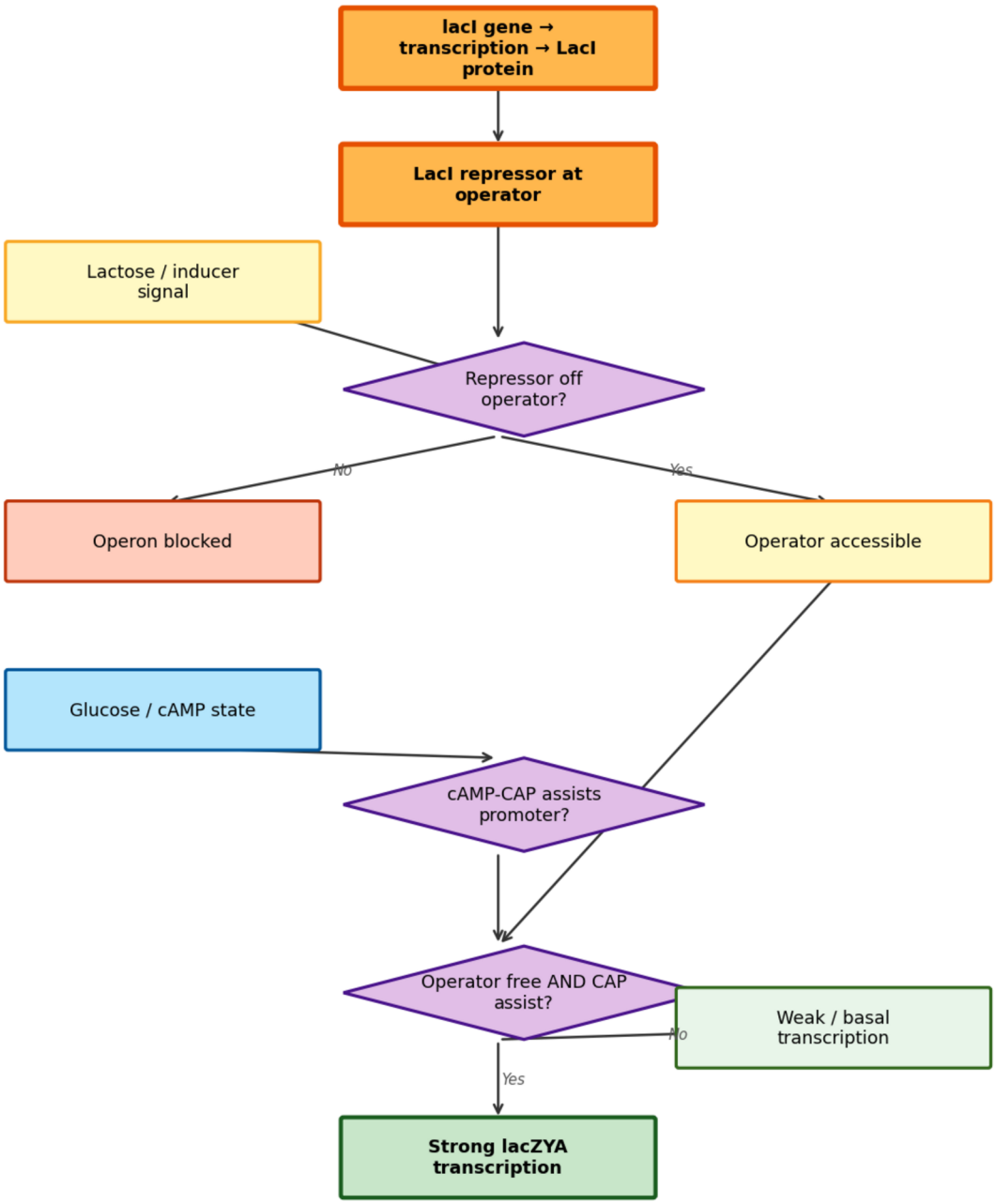
5.3 Hybrid diagram (literature logic + RegulonDB entity completeness)

The **hybrid** merges interpretive structure from reviews and LLM drafts with **database-grounded entities**. The key addition is an explicit step for ***lacI* transcription to LacI protein** upstream of operator control - a node that matters for perturbations such as tuning repressor dosage via CRISPRi. Boolean-style gates are retained from the literature-first chart.

Exactly one structural addition vs. Figure A: the two orange nodes at the top are the RegulonDB-class spine that Figure A's red annotation warned was missing. Purple diamonds and the green transcription outcome use the same color language as Figure A; only the orange chain is new.

Figure C. Hybrid. Orange = explicit *lacI* gene to protein (RegulonDB-style completeness). Purple diamonds = Boolean AND gates (same convention as Figure A). Green = strong lacZYA transcription output.

Figure C — Hybrid diagram
Orange = explicit lacI gene→protein (RegulonDB-style) | Purple = Boolean AND gates | Green = strong output



5.4 At-a-glance: literature-first vs. hybrid

Feature	Literature-first (Fig. A)	Hybrid (Fig. C)
lacI gene to LacI protein	**Absent** (repressor assumed)	**Present** as two orange nodes
Explicit AND gate	Yes (purple diamond)	Yes (same convention)
Perturbations readable from picture	Nutrients, operator, CAP branch	Those **plus** *lacI* expression as its own lever
RegulonDB entity completeness	Not the goal	Explicitly merged

Interactive multi-encoding views are available at the GLMP demo viewer: https://storage.googleapis.com/regal-scholar-453620-r7-podcast-storage/glmp-v2/viewer_demo/glmp-viewer-demo-v1-v6.html?process=ecoli_lac_operon&version=v1

---

6. Why the two-input induction condition is not automatic in database-native views

Strong induction requires relief of LacI-mediated repression *and* favorable catabolite control (glucose scarcity to sufficient cAMP for CRP). Textbook diagrams express that as **AND** logic. Curated regulatory databases excel at listing regulators, targets, and evidence; they do not always export that **simultaneous** requirement as explicit gate nodes. LLM- or review-driven charts supply that interpretive layer - subject to validation - while RegulonDB-class edges supply **who is connected to whom**.

---

7. Layered hybridization (recommended workflow)

1. **Regulatory backbone:** TF to gene resolution from a trusted resource (for *E. coli*, RegulonDB or equivalent).
2. **Interpretive overlay:** literature or LLM-assisted gates and feedback arcs where they clarify conditionality.
3. **Parts audit:** EcoCyc / BioCyc for enzyme identities and reactions.4
4. **Identifier layer:** KEGG or model-organism locus tags attached as node metadata after topology stabilizes.5

Naive union of every node type from multiple ontologies typically **obscures** the very branch logic that makes diagrams useful for perturbation planning.

---

8. Practical guidance: prompt discipline and operational habits

The same pathway, same organism, and different natural-language instructions yield different Mermaid topologies. Best practice: generate two charts, diff the source, reconcile against one primary review figure or database page, and archive both the Mermaid text and the citation in supplementary material.

More broadly, investigators benefit from routinely pairing literature search, structured databases, and diagram-as-code so that **inputs, branch points, and feedback** are explicit before large

experiments or model training runs. Public GLMP galleries illustrate how source mixtures change charts; they are optional references, not prerequisites.

---

9. Evidence buckets for perturbation design

1. Primary literature and reviews (mechanism, conditions).
2. Pathway and interaction databases (Reactome, KEGG, WikiPathways, BioCyc; STRING for functional linkage).6-8
3. Gene-centric resources (NCBI Gene, UniProt, Ensembl, GO).
4. Regulatory layers (RegulonDB, Abasy-class summaries; refs. 3, 9).
5. Perturbation execution (reagents, libraries, chemistry).
6. Data archives for realistic readouts (e.g. GEO).10

---

10. What logic charts add on top of databases and papers

Typical resources	Logic-style flowchart adds
Gene lists and canonical pathway maps	Explicit branching and AND/OR reasoning under stated conditions
Static lookup-optimized maps	"If this edge is removed, what fails first?"
Perturbation methods	Menu of informative interventions tied to mechanism sketch

---

11. Synthesis

Databases summarize what is connected; papers record what was measured under stated designs; a logic flowchart helps investigators choose perturbations that disambiguate mechanisms and anticipate qualitative directions before committing full experimental or computational cost.

Acknowledgments and AI Use Disclosure

The author used Claude (Anthropic, claude-sonnet-4-6) to assist with programmatic rendering of Figures A-C via Python/matplotlib and for PDF/DOCX typesetting. All scientific content, diagram structure, biological interpretation, and conclusions are the author's own. The GLMP viewer diagrams referenced in §5 were generated via Cursor IDE using various large language models; visual differences between the two sets illustrate the prompt and tool sensitivity discussed in §8.

Data and code availability

Public GLMP process JSON and viewers reside on Google Cloud Storage. The lac operon multi-viewer:

https://storage.googleapis.com/regal-scholar-453620-r7-podcast-storage/glmp-v2/viewer_demo/glmp-viewer-demo-v1-v6.html?process=ecoli_lac_operon&version=v1

No new experimental data were generated for this methods draft.

Figures A-C in this submission were rendered programmatically for publication using Python/matplotlib. The GLMP viewer diagrams referenced in §5 were generated via Cursor IDE using various large language models; visual differences between the two sets illustrate the prompt and tool sensitivity discussed in §8.

References

1. Jacob F, Monod J. Genetic regulatory mechanisms in the synthesis of proteins. *J Mol Biol.* 1961;3(3):318-356.
2. Mermaid documentation. https://mermaid.js.org/ (accessed 2026).
3. Gama-Castro S, et al. RegulonDB: a database of transcriptional regulation in *Escherichia coli* K-12. *Nucleic Acids Res.* https://regulondb.ccg.unam.mx/
4. Keseler IM, et al. EcoCyc: enriching the BioCyc collection of databases. *Nucleic Acids Res.* 2021;49(D1):D608-D612.
5. Kanehisa M, et al. KEGG for taxonomy-based analysis of pathways and genomes. *Nucleic Acids Res.* 2023;51(D1):D587-D592.
6. Gillespie M, et al. The Reactome Pathway Knowledgebase 2024. *Nucleic Acids Res.* 2024;52(D1):D672-D678.
7. Martens M, et al. WikiPathways: connecting communities. *Nucleic Acids Res.* 2021;49(D1):D613-D621.
8. Szklarczyk D, et al. The STRING database in 2025. *Nucleic Acids Res.* 2025;53(D1):D638-D646.
9. Escorcia-Rodriguez JM, Tauch A, Freyre-Gonzalez JA. Abasy Atlas v2.2. *Comput Struct Biotechnol J.* 2020;18:1228-1237.
10. Barrett T, et al. NCBI GEO: archive for functional genomics data - updated. *Nucleic Acids Res.* 2013;41(D1):D991-D995.
11. Nielsen AJ, Der BS, Shin J, Vaidyanathan P, Densmore D, Paralanov V, Strychalski EA, Ross D, Voigt CA. Genetic circuit design automation. *Science.* 2016;352(6281):aac7341. https://doi.org/10.1126/science.aac7341

Zenodo DOI: https://doi.org/10.5281/zenodo.20831781
Suggested category: Methods / Systems biology / Synthetic biology

Correspondence: gwelz@gc.cuny.edu